

Запускать так: `./run.sh`

По необходимости доставить библиотеку с решателем ЛП (в решении есть только солвер непрерывной ЛП, не целочисленной, без дорешки)

`sudo apt install coinor-libclp-dev`

Жадник

Глобально хочется написать следующий алгоритм: сопоставим людям магазины в которые они должны идти, после чего будем уменьшать целевую функцию меняя целевой магазин для части людей на другой. Для этой идеи зададим такое условие:

Опустим факт того, что магазины надо открывать, а дадим возможность людям выбрать в какой магазин они хотят пойти и откроем все те, которые нужно хотя бы одному человеку, то есть сформулируем следующее:

$$\min_{(x_1, x_2, \dots, x_M) \in \{1, 2, \dots, N\}^M} \sum_{i=1}^M \text{dist}(\text{guy}_i, \text{store}_{x_i}) + \sum_{j=1}^N \text{cost}_j \cdot \text{is_open}_j,$$

где выполнено $\sum_{i: x_i=j} \text{need}_i \leq \text{capacity}_j$ и $\text{is_open}_j = I(\exists i : x_i = j)$.

Это можно переписать в линейную программу, которой мы однако пользоваться не будем. Для этого заменим $x_1, x_2, \dots, x_M$ на индикаторы того, что i -ый человек выбрал магазин j . Получим следующее:

$$\min_{\substack{\{x_{i,j}\}_{i=1, \dots, M; j=1, \dots, N} : x_{i,j} \in \{0,1\} \\ \{\text{is_open}_j\}_{j=1 \dots N} : \text{is_open}_j \in \{0,1\}}} \sum_{i,j} x_{i,j} \text{dist}(\text{guy}_i, \text{store}_j) + \sum_j \text{cost}_j \cdot \text{is_open}_j,$$

где выполнено $\forall i = 1, \dots, M \sum_{j=1}^N x_{i,j} = 1; \forall j = 1, \dots, N \sum_{i=1}^M \text{need}_i \cdot x_{i,j} \leq \text{capacity}_j \cdot \text{is_open}_j$.

Сперва напишем эвристику для ответа. Для всех людей отсортируем магазины в порядке увеличения длины до них. Рассмотрим некоторую перестановку людей. В порядке этой перестановки назначим человеку ближайший магазин, куда он может ещё пойти, то есть ближайший, остаточная вместимость которого больше или равна потребности человека.

На удивление эта эвристика уже сумела пройти первый тест на одной случайной перестановке. Метрики составили

3276471.40, 178669106.40, 5453881.94, 38438142.55, 10849389.49, 11969284.22

и это дало 3 балла как 3, 0, 0, 0, 0, 0.

Количество итераций у генерации перестановки было повышено до 10000, это не дало результатов, метрики составили

3276471.40, 178669106.40, 5413828.12, 38371121.06, 10803075.22, 11896122.65.

По этой причине количество итераций было уменьшено до 1, чтобы получить хоть какое-нибудь жадное решение.

Далее были написан 1-opt, который делал следующее:

Посмотрим на произвольного человека и заставим его пойти в другой магазин, а именно во все магазины по очереди и взглянем в каком случае метрика станет наименьшей. Если

это не тот магазин, в который он ходит сейчас, то примем это изменение. Так будем делать, пока есть к кому применять изменения. До сходимости на каждом из тестов было проделано соответственно 1, 14, 26, 29, 220, 482 итерации. Метрики оказались следующими:

3269821.32, 154082909.65, 5167171.45, 37165227.47, 9552597.41, 8639293.94.

Это соответствует баллам 5, 0, 0, 0, 3, 3.

Далее был добавлен 2-орт:

Рассмотрим две двух людей которые ходят в разные магазины, возможно они хотели бы ходить наоборот, но 1-орт это не сумел, поскольку ни один из магазинов не может принять их обоих одновременно. Попробуем поменять их местами, если это уменьшит метрику, то примем такое изменение. Снова отработаем до сходимости.

Эта поправка не прошла больше тестов, зато улучшила метрики. Количество итераций до сходимости по тестам составило 0, 0, 55, 111, 78, 74, метрики составили

3269821.32, 154082909.65, 5137255.90, 37061561.87, 9468770.90, 8586540.59.

Визуализировав нынешнее решение наблюдается проблема, что несколько человек требовали открыть им какой-то магазин и стоимость открытия полупустых магазинов гораздо выше, чем трудность пройти в другой магазин. Более того, среди открытых магазинов походы людей скорее похожи на кластеризацию по ближайшему магазину, что в целом похоже на истинно верное разбиение. Попытаемся исправить проблему открытия лишних магазинов: по очереди попробуем закрыть каждый магазин и отправить всех его посетителей в другие магазины старым способом. Если метрика станет хуже или нам это не удастся, то вернём как было. Это улучшило метрики, они составили

3269821.32, 31177877.16, 4792800.70, 28969918.34, 9153974.36, 7933857.27.

Что дало 17 баллов 5, 0, 3, 3, 3, 3. Также было попробованно пытаться закрыть раньше те магазины, в которых больше свободного места, то есть те, которые не понятно зачем открывать, потому что их потенциал далеко не раскрыт. Это улучшило метрики:

3269821.32, 23960419.05, 4801888.85, 28628889.58, 9195056.99, 7937979.43.

20 баллов: 5, 3, 3, 3, 3, 3.

Причём решение всё также похоже на то, что человек идёт в ближайший из открытых магазинов. Так что возможно достаточно просто открыть магазины правильно, а люди имеющимися операциями перейти в магазин получше или поменяться двум людям сумеют сами пойти туда, куда лучше. Если это свойство решение начнёт нарушаться тогда и подумаем про его реализацию.

Генетика

В постановке на конец первого подхода задача буквально звучит как назвать правильную битовую строку из 0 и 1 как именно надо открывать магазины. Такая постановка идеально подходит для генетических алгоритмов, поскольку именно по этой битовой строке можно и писать генетику.

Сперва напишем генератор популяции, для этого надо найти много валидных решений. В валидном решении может быть довольно разное количество объектов, а потому примерное количество открытых магазинов возьмём как нынешний `best_ans`. А именно с настраиваемой вероятностью изменим каждый конкретный бит его строки взятия магазинов. На четвёртом тесте возникла проблема, что популяция очень долго не генерировалась, из-за того,

что OneTwoOpt применялся слишком долго. Из-за этого было принято решение, что если популяция генерируется дольше 15 секунд, то эта эвристика будет отключаться.

Напишем скрещивание. Делать будем банально: возьмём общие единички двух родителей, а из необщих выберем половину случайно. Если получилось невалидное решение то добавим ещё несколько случайно. Также было решено добавить лучшее решение в популяцию.

Отбор будет в формате соревнования на выбывание. Сколько-то лучших особей возьмём сразу. Среди оставшихся, пока их слишком много будем брать произвольную пару и того, у которого метрику хуже — убивать.

Вырожденной будем считать популяцию, если среднее хэминговых расстояний упало ниже какого-то процента от всей длины.

Настроим параметры, в данный момент есть параметры: размер популяции, количество скрещиваний, количество автоматически проходящих в следующий тур, вероятность начального шума при генерации и процент для вырожденности. Сперва выберем размер популяции:

Размер популяции	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
6	3269821.32	23294909.40	4794642.39	28551967.99	9178228.06	7928423.47
12	3269821.32	23196013.73	4796617.85	28619017.75	9205347.88	7926551.06
20	3269821.32	23153940.28	4799456.82	28556793.12	9183020.72	7952878.71
40	3269821.32	23108194.36	4794446.11	TL	9166215.09	7941635.27
60	3269821.32	23091644.68	4797463.14	TL	9203246.91	7942354.57

В целом метрики особо не меняются, только метрика второго теста немного улучшается. При этом время растёт, возьмём что-то небольшое, например 8.

Теперь выберем насколько случайно генерировать популяцию (сейчас в какой-то степени это генетика с мутацией, так как локальный поиск может менять маску, без этого алгоритм сходится куда медленнее).

p изм. бита	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.10	3269821.32	23857643.76	4795296.52	28696482.03	9199272.88	7928665.98
0.30	3269821.32	23376261.13	4796120.24	28540332.69	9168195.75	7950829.37
0.50	3269821.32	23432724.85	4794196.76	28606492.10	9175103.50	7910950.80
0.75	3269821.32	23248569.03	4787704.73	28523441.07	9145309.49	7910119.39
1	3269821.32	23629279.19	4800240.89	28576818.73	9175638.48	7927473.66

Лучшие метрики вышли при 75 процентах.

Теперь выберем число детей.

число детей	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
2	3269821.32	23300451.09	4795831.31	28560029.07	9173743.15	7950573.86
4	3269821.32	23966514.26	4792624.10	28555236.93	9176602.43	7939502.74
6	3269821.32	23478215.22	4792443.38	28640927.67	9184223.48	7952821.64
8	3269821.32	23080182.74	4803200.28	28554890.42	9194702.85	7925975.45
12	3269821.32	23236797.36	4790673.84	28621952.35	9170924.55	7941166.29
20	3269821.32	23451110.75	4796777.04	TL	9173469.00	7920553.31

Оптимальное 8. Остаётся выбрать сколько лучший сразу проходят без турнира:

Метрики от неё практически не изменяются и скорее всего это погрешность от вероятностных шагов. Поставим 0, чтобы убрать непонятную механику.

всегда выживают	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0	3269821.32	23170732.92	4799950.65	28635275.02	9184561.08	7948385.01
2	3269821.32	23365566.26	4798942.40	28700063.19	9157640.68	7938723.78
4	3269821.32	23499924.74	4795235.87	28591610.52	9163283.58	7936070.32
6	3269821.32	23263392.61	4798446.69	28579868.73	9147024.72	7951352.45

Однако даже после настройки генетика не особо добивается успеха. Метрики не особо изменились по сравнению с предыдущим результатом.

Нынешнее решение уже насытило все магазины достаточно сильно и хотелось бы просто выбрать магазины лучше. Генетика же не может рассмотреть маленькие изменения, потому как работает с очень маленьким суммарным покрытием. Попробуем добавить какую-то мутацию, чтобы затрагивать как можно больше магазинов.

Попробуем самую простую мутацию для наших целей: добавим несколько магазинов в маску и пересчитаем метрику. При добавлении случайных битов метрики составили:

3269821.32, 23081307.85, 4792545.34, 28570234.24, 9161673.13, 7929047.19.

То есть всё так же поменялись на шум.

Видимо имеющееся решение в целом близко к генетическому оптимуму и магазины стоит выбирать тоньше. Глобально решение правдоподобное и метрики надо улучшать маленькими аккуратными улучшениями.

Локальный поиск и Tabu search

Отключим генетику и попытаемся написать локальный поиск на смене магазинов, чтобы найти наилучшее множество, зная, что нынешний ответ в целом близок к истинному.

Первая операция, которую мы попробуем применить это выбрать вместо уже имеющегося магазина какой-то другой неподалёку. Для этого закроем старый, откроем новый неподалёку, скорее всего люди не против перейти в него, назначим их в открытый магазин, а после для улучшения качества дадим людям выбрать магазин получше операцией из жадника. Если метрика лучше, то примем изменение, иначе откатим и пойдём дальше.

От такого подхода метрики составили:

3269821.32, 22832384.97, 4741160.74, 27401816.46, 8973929.43, 7556449.52

Что взяло баллы 5, 3, 3, 3, 3, 3, то есть те же, что и раньше, однако перебило метрики генетики. В целом всем метрикам не хватает процента до большого порога, так что остаётся написать что-то небольшое.

Однако пока есть проблема, что алгоритм долгий и на последних трёх тестах тормозится по времени. Попытаемся его ускорить через tabu search. Если мы недавно смотрели на какой-то магазин, а его окрестность за последний шаг не менялась, то можно не пытаться рассмотреть его снова. Реализуем это так: заведём на конкретный вид хода таймер, если ход не удался, то запретим пробовать такой ход на какое-то время. Чтобы узнать какое надо время взглянём на количество принятых шагов в нынешнем алгоритме.

	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
операций принято	0	8	22	108	142	293
операций всего	50	199	1582	3690	7295	3767

В целом принимающих операций не много и после 200 магазинов резко возрастает, так что установим табу-время как константу в зависимости от размера.

Возьмём максимальный параметр при котором метрики упадут не сильно:

время запрета	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
2, 10	3269821.32	22839415.78	4735784.81	27422503.61	8977293.08	7543504.08
3, 15	3269821.32	22839010.27	4737756.03	27509832.26	8963518.64	7562266.56
4, 20	3269821.32	22836140.73	4743347.87	27524114.46	8978058.42	7548091.56
5, 25	3269821.32	22827669.99	4751856.66	27427461.18	8968238.96	7552683.94

Порога в четверть времени всё равно мало чтобы ухудшить метрики, видимо повторы в целом не важны, просто для каждой пары попробуем по разу. Такой подход даёт метрики:

3269821.32, 23078966.70, 4758709.93, 27746396.57, 9018617.48, 7535582.74

Всё же отличимо хуже от большого табу. Вероятно проблема в том, что есть хорошее изменение, которое применялось несколько раз, потому что магазины перебегали по цепочке через промежуточный. Чтобы такого не было вероятно надо повысить размер окрестности.

размер окрестности	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
10	3269821.32	23078966.70	4758709.93	27746396.57	9018617.48	7535582.74
30	3269821.32	22947826.50	4750659.63	27743935.00	8982870.48	TL
50	3269821.32	22963382.36	4755203.35	27586400.67	8998578.32	TL

При 50 метрики уже возвращаются, но время лучше коротало табу, так что оставим большое табу.

Другая возможность, что кроме магазина, который надо было открыть неподалёку, что возможно сейчас какие-то люди ходят слишком далеко, а открой мы магазин поближе, то они бы сэкономили много расстояния. Для этого в магазин надо отправлять каких-то людей, которым это выгодно. Будем добавлять магазины по одному, фактически надо понять можно ли отправить в магазин людей так, чтобы их сэкономленное расстояние было больше, чем стоимость открытия магазина. Это фактически рюкзак, где стоимость — экономия расстояния. Его нам вероятно хватит решать очень неточно, просто посчитаем отношение потребности к улучшению и возьмём жадно.

Метрики изменились на шум, этот шаг не приносит большого профита. Значит скорее всего открывать минимум магазинов правильным образом — хорошая идея, однако в нынешней реализации она уже съедает всё время. Надо открыть максимально хороший набор магазинов за сравнительно быстрое время.

Отжиг

Операция, предложенная в локальном поиске была хорошая и улучшила метрики, однако занимает много времени, чтобы пытаться менять по 2 магазина, менять цепочки или делать нечто подобное. Естественный способ сэкономить время и прийти в оптимум по большому пространству — отжиг, его и напишем.

Чтобы сэкономить время на большее число шагов будем добавлять магазин грубее: просто добавим магазин и приближением рюкзака добавим в него людей, в целом за счёт освоившегося места оптимум должен улучшаться 1,2-орт ом. Однако они занимают много времени, так что попробуем посмотреть на упрощённый вариант.

Изменение скорее всего локальное, то есть стоит пытаться найти место получше только в близкие магазины, которые освободились. Возьмём только людей, которые ходят в эти магазины.

Метрики от такого подхода составили:

3269821.32, 23963958.81, 4739063.31, 27498100.06, 8988694.40, 7668690.78

Второй тест на удивление умер, в остальном метрики получились сопоставимые с локальным поиском.

Возможно одна из сильных сторон локального поиска была в том, что изменение пыталось примениться ко всем возможным шагам, а случайные шаги не попали в какие-то из них. Попробуем написать отжиг на полном обходе всех ходов.

У такого отжига метрики стали значительно хуже:

3269821.32, 23241979.35, 4757312.65, 28633365.26, 8984290.23, 7629180.47.

Проблема со вторым тестом вероятно заключалась в том, что на нём сложно найти замену, которая допускает какую-либо расстановку покупателей и потому взятие случайной мешало найти какой-либо допустимый ход.

В целом отжиг не улучшает метрики, на имеющихся операциях окрестность недостаточно велика, чтобы получить лучшие метрики. Либо искомые метрики в целом не лежат в этой окрестности.

Назад в локальный поиск

Из имеющихся окрестностей мы вероятно выбрали максимально хорошее значение, вопрос в том, что окрестность недостаточно велика.

В целом замена одного магазина на один другой нам уже доступна, скорее всего какая-то замена была разрешена, но мы не нашли валидное решение. Попробуем переписать 1, 2-орт из жадника. В нынешнем решении разрешаются только жадные вставки, однако возможно надо вытеснить одного клиента другим и повторить попытку.

Так и напишем, чтобы количество людей на починку не росло разрешим при поиске нового места удалить 1 из текущих покупателей и перезапуститься. Минимизировать будем пытаться оставшийся demand, потому как маленькую потребность скорее всего проще положить в какой-то из магазинов.

Простое добавление с вытеснением дало примерно те же метрики:

3269821.32, 22832218.06, 4740035.18, 28258944.09, 8980581.40, 7815567.42.

Возможно также важен порядок добавления. Попробуем выделить всех тех покупателей, которые не нашли магазин и начинать с них. Метрики составили:

3269821.32, 22826639.01, 4734306.78, 27394823.67, 8953879.06, 7425750.61(TL)

В целом снова отличие на шум. Однако был взят шестой тест, но это произошло по грани и не уложилось во время, потратив 20 минут.

Возможно стоит распахивать не самым жадным образом, а всё таки попытаться искать лучший из вариантов. Полный обход дерева получил TL на всех тестах, так что попробуем что-то более быстрое.

Beam Search

Попробуем всё таки реализовать поиск наилучшей вставки при разрешённых вытеснениях. Чтобы не получить TL попробуем BeamSearch. Будем искать лучшую ветку, поддерживая несколько возможных обменов.

Будем пробовать следующее: берём покупателя и пытаемся добавить его в один из открытых магазинов, если в какой-то из них он не вмещается удаляем какого-то покупателя из магазина и пытаемся переназначить его. Изменим также и план удаления.

Удалять будем несколько покупателей различного вида:

- покупателя с минимальным demand для удаления
- покупателя, которого достаточно удалить, у которого максимальная разница между его нынешним магазином и ближайшим
- покупателя, которого достаточно удалить, у которого максимальная разница между его нынешним магазином и открытым ближайшим.

Также для неполных решений надо придумать метрику того, насколько они плохие, так, чтобы верить в их улучшение, но они вытесняли неполные решения. Попробуем предположить сколько примерно они прибавят расстояния после всех добавлений до валидного решения, а именно наверное каждый из них переместится чуть дальше, кроме того выгонет кого-то другого. Посмотрев глазами на решения покупатель редко ходит дальше, чем в третий по близости из открытых магазинов. Пока попробуем в самом начале проинициализировать штраф как расстояние до третьего открытого магазина, в конце подберём исправляющую константу.

Теперь OneTwoOpt можно заменить на удаление одного человека и поиск ему нового места через замены BeamSearch. Смена магазина — как и раньше, это закрытие магазина и вызов поиска нового места для каждого из его бывших посетителей. Добавление магазина — добавим магазин и вызовем OneTwoOpt, если метрика стала лучше, то оставим. Удаление магазина: удалим магазин, вызовем OneTwoOpt для всех его посетителей, если удалось их распределить и метрика стала лучше, то оставляем.

Такой подход до настройки также не изменил метрики. Настроим чуть позже, а пока попытаемся расширить спектр шагов. Наибольший скачок всё ещё даёт BeamChangingStore который меняет 1 магазин на другой. Остальные нужны для того, чтобы можно было получить решение даже если нынешнее количество магазинов неверное. Посмотрим на другую операцию: разрешим также добавлять не одинаковое количество магазинов: можно добавить от 1 до 2 магазинов и удалить от 1 до 2 магазинов. Причём всё они должны быть близко друг к другу.

Для этого сделаем такую операцию: возьмём магазин который мы планируем добавлять, добавим его, возьмём его соседа на удаление, соседа удаляемого на добавление и соседа добавляемого на удаление. Причём соседи смогут совпадать, так что все 4 комбинации могут получиться.

Эта операция начала брать ещё и шестой тест, метрики составили:

3269821.32, 22853282.23, 4743297.52, 27536127.65, 8972549.07, 7379278.01

22 балла: 5, 3, 3, 3, 3, 5.

Попробуем настроить beam search. Параметры у нас следующие: глубина поиска, количество поддерживаемых экземпляров в beam search, количество лучших кандидатов каждого из трёх типов и количество людей и магазинов считаемых близкими.

depth	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
3	3269821.32	22853482.29	4755148.27	27347130.77	8983229.43	7406927.20
6	3269821.32	22874182.67	4741744.29	27722313.88	8981047.51	7401300.28
9	3269821.32	22946688.03	4744884.29	27697091.75	8969247.62	7414873.75
12	3269821.32	23094508.09	4749557.33	27960226.35	9000362.93	7447148.25

Начнём с глубины поиска, чем больше тем медленнее, но больше шанс найти улучшение.

На 9 и 12 метрика уже всегда не наилучшая, там скорость всегда важнее, чтобы успеть больше ходов, а между 3 и 6 зависит от теста. Поставим 6 как вариант получше для 3,5,6 тестов.

Теперь количество поддерживаемых решений в бимме:

beam_const	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
4	3269821.32	22840160.82	4748537.49	27617617.38	8975771.67	7405003.69
8	3269821.32	22841609.47	4741770.17	27724921.77	8979174.35	7425901.35
12	3269821.32	22918210.26	4754738.62	27757644.36	8958985.82	7422930.7
16	3269821.32	22964651.54	4761082.70	27764480.70	8984225.35	7417389.60

На 12 и 16 метрики снова хуже кроме 5 теста, для 8 метрики тоже хуже чем для 4, однако это не оставит свободы в выборе видов соседей, так что возьмём 6.

Остаётся определиться с количеством соседей каждого типа, попробуем взглянуть какой из них даёт больший прирост и такого и поставим больше.

ld/mcl/mopcl	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
4/0/0	3269821.32	22837608.12	4741975.41	28020827.41	8981438.17	7472101.10
0/4/0	3269821.32	22833901.76	4742793.78	27802313.91	8949818.95	7401942.08
0/0/4	3269821.32	22842413.62	4745366.06	27671113.84	8937927.45	7386533.40
2/2/0	3269821.32	22903739.69	4739671.95	27808140.09	8950010.24	7400011.37
2/0/2	3269821.32	22847944.93	4740147.60	27632761.73	8976779.43	7417488.93

Лучшие позиции распределились хаотично, возьмём поровну — 2/2/2.

В целом настройка не помогла, метрики примерно те же. Разве что первый шаг — это не то, что нужно шестому тесту. Он существенно лучше от 4/0/0 к 0/0/4.

Интуиция на счёт локальных операций как-то не работает, попробуем что-то что на интуицию не опирается.

Линейные релаксации

Как мы уже поняли маска (выбранные магазины) достаточно хорошо приблизилась, однако и локальный поиск, и генетика приходят не туда и в добавок зашумлены. Попробуем написать что-то менее шумное для фиксированного набора магазинов, чтобы иметь более чёткое понимание о том, что делает тот или иной ход для магазинов.

При фиксированных магазинах цена открытия не меняется и надо лишь определиться с тем куда кому идти. Рассмотрим естественную линейную релаксацию: заведём переменные $x_{i,j}$ — i -ый человек ходит в j -ый магазин. Тогда ограничения понятны: $\sum_i x_{i,j} \leq dem_j$ и $\sum_j x_{i,j} = 1$ и минимизировать надо $\sum_{i,j} dist_{i,j} x_{i,j}$. Причём если все $x_{i,j}$ оказались целыми, то это и есть решение. Пока возьмём маску из жадника и посмотрим сколько в ней нецелых переменных.

Сперва зафиксируем тех, кто уверен в правильности их выбора, то есть в линейной релаксации стоит чёткое 1. После чего, будем пытаться добавить человека в тот магазин, в который у него была большая переменная, в порядке убывания переменных по всем, чтобы почти 1 было реализовано по возможности. Среди тех, кто не будет назначен попытаемся сделать вставку через beam search. В целом до неё на жаднике дошло не очень много вершин: 0, 0, 7, 36, 41, 115 соответственно.

Теперь под это напомним старую операцию изменения маски: меняем биты на соседние, если улучшается, то применяем, если ухудшается, то не применяем. Как и раньше закончим на шаге с изменением двух битов. Метрики на нём составили:

3269821.32, 23040516.31, 4749343.29, 28333957.54, 9125075.37, 7942559.30.

Метрики плохие, при том, что за нас почти всё сделала линейная релаксация, значит наша интуиция по поводу того какие маски надо перебирать — отвратительная и мешает нормальной программе работать. Напишем что-то от нас не зависящее.

Обходы дерева масок

Пойдём обходить деревья всех масок не думая что делать логично, а что не логично. Посмотрим на маску как на битовый вектор, а на вычисление метрики как на чёрный ящик, который наверное похож на непрерывный. Рассмотрим операции замены некоторого бита и попытаемся написать хороший обход дерева.

Beam Search

Напишем тривиальный Beam Search: менять произвольный бит, заведём хэш таблицу всех битвекторов которые мы уже раскрывали. Сменим по 1 бит в случайных позициях, если такой вектор уже был, то просто попробуем с другим битом. У такого решения 2 параметра: количество поддерживаемых копий у beam search и количество решений, которые мы получаем из каждого вектора. Опустим второе и просто прочередуемся в каждом бите.

Также было добавлено маленькое увеличение по скорости: есть суммарная demand открытых магазинов не хватает, то и запускаться не будем, не реализуется.

beam const	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
3	3269821.32	22727090.20	4719772.60	TL	TL	TL
5	3269821.32	22727090.20	4726579.41	TL	TL	TL

TL это конечно плохо, но от больших тестов мы находимся в соток доле процента. Учитывая то, что для больших нормально справлялся Beam Search, то в рамках этого метода попытаемся дожать метрику на маленьких тестах.

Учитывая то, что разрыв минимальный, то возможно её можно выжать из уже имеющейся маски аккуратно дожав её. Реализуем через ЛП полный 3-opt, а именно для всех троек магазинов разрушим всех её покупателей и переназначим самым выгодным образом согласно ЛП релаксации. Если битов больше 30, то мы получим TL от слишком большого числа троек и запустим только для двоек.

Метрики на первых трёх тестах составили

3269821.32, 22724525.23, 4716885.78.

Два больших порога и один маленький. Причём написанная операция скорее эффективна при небольшом количестве магазинов в ответе, при большом же не принесла выгоды. Для мелких ответов не понятно что делать.

Итог

Лучшие метрики получаются склейкой решений: пока магазинов мало (до 200) хорошо работает полный beam search, когда их становится больше, то тот уходит в TL и наилучшие метрики у Beam Search с выталкиванием. У такой склейки решений метрики составили

3269821.32, 22724525.23, 4721234.11, 27362241.03, 8911004.70, 7389457.27

Что дало 24 балла 5, 5, 3, 3, 3, 5.